Respostas dos Exercícios de Banco de Dados — Subconsultas, CTEs e Views

1- Exercício Proposto 1: Subconsultas e quantificadores

a) Liste produtos com preço acima da média geral:

```
SELECT id_produto, nome, preco
FROM produto
WHERE preco > (SELECT AVG(preco) FROM produto);
```

b) Localize clientes com ao menos um pedido pago usando IN:

```
SELECT id_cliente, nome
FROM cliente
WHERE id_cliente IN (
  SELECT id_cliente
  FROM pedido
  WHERE status = 'Pago'
);
```

c) Reescreva a busca usando EXISTS:

```
SELECT c.id_cliente, c.nome
FROM cliente c
WHERE EXISTS (
  SELECT 1
  FROM pedido p
  WHERE p.id_cliente = c.id_cliente
    AND p.status = 'Pago'
);
```

d) Encontre clientes sem pedidos usando NOT EXISTS:

```
SELECT c.id_cliente, c.nome
FROM cliente c
WHERE NOT EXISTS (
```

```
    SELECT 1
    FROM pedido p
    WHERE p.id_cliente = c.id_cliente
);
```

e) Compare ANY e ALL com os preços da categoria LIVRO:

```
-- Exemplo com ANY (retorna produtos com preço maior que P3LO MENOS UM livro):

SELECT nome, preco
FROM produto
WHERE preco > ANY (
    SELECT preco FROM produto WHERE categoria = 'LIVRO'
);
-- Exemplo com ALL (retorna produtos com preço maior que TODOS os livros):

SELECT nome, preco
FROM produto
WHERE preco > ALL (
    SELECT preco FROM produto WHERE categoria = 'LIVRO'
);
```

Comparação Conceitual: O quantificador ANY compara o valor da consulta principal com cada valor individual retornado pela subconsulta e retorna VERDADEIRO se ao menos uma comparação for atendida (equivale a ser maior do que o preço do livro MAIS BARATO). Já o quantificador ALL exige que a condição seja estritamente satisfeita para TODOS os valores da subconsulta simultaneamente (equivale a ser maior do que o preço do livro MAIS CARO).

2- Exercício Proposto 2: Correlação e CTEs

a) Liste produtos acima da média da própria categoria:

```
SELECT p1.id_produto, p1.nome, p1.categoria, p1.preco
FROM produto p1
```

```
WHERE p1.preco > (
  SELECT AVG(p2.preco)
  FROM produto p2
  WHERE p2.categoria = p1.categoria
);
```

b) Recupere o pedido mais recente de cada cliente:

```
SELECT p1.id_pedido, p1.id_cliente, p1.data_pedido, p1.status
FROM pedido p1
WHERE p1.data_pedido = (
  SELECT MAX(p2.data_pedido)
  FROM pedido p2
  WHERE p2.id_cliente = p1.id_cliente
);
```

c) Reescreva uma correlação usando JOIN com agregação prévia:

```
SELECT p.id_produto, p.nome, p.categoria, p.preco
FROM produto p
INNER JOIN (
  SELECT categoria, AVG(preco) AS preco_medio
  FROM produto
  GROUP BY categoria
) media_cat ON p.categoria = media_cat.categoria
WHERE p.preco > media_cat.preco_medio;
```

d) Crie uma CTE para calcular faturamento por produto:

```
WITH faturamento_produto AS (
  SELECT
    p.id_produto,
    p.nome AS nome_produto,
    SUM(ip.quantidade * ip.preco_unitario) AS faturamento_total
  FROM produto p
  INNER JOIN item_pedido ip ON p.id_produto = ip.id_produto
```

```
    GROUP BY p.id_produto, p.nome
)
SELECT id_produto, nome_produto, faturamento_total
FROM faturamento_produto
ORDER BY faturamento_total DESC;
```

e) Compare clareza, cardinalidade e provável custo das versões:

Análise Comparativa:
1. Clareza: CTEs e JOINs com subconsultas agregadas são substancialmente mais legíveis e fáceis de dar manutenção do que subconsultas correlacionadas aninhadas no WHERE, pois separam a lógica de cálculo da lógica de seleção principal.
2. Cardinalidade: Subconsultas em clausulas WHERE comparativas exigem retorno estritamente de 1 valor escalar por linha processada (cardinalidade 1:1), ao passo que JOINs com CTEs/subconsultas agregadas consolidam a cardinalidade do grupo no JOIN (ex.: 1 linha por categoria).
3. Provável Custo Executivo: Subconsultas correlacionadas tendem a ter custo elevado O(N * M) porque a subconsulta pode ser reexecutada para cada linha da tabela externa. Já as CTEs e o JOIN com agregação prévia pré-calculam as médias/totais apenas uma vez (O(N) + custo do JOIN), otimizando radicalmente o plano de execução do banco de dados.

3- Desafio Integrador: Entregue consultas avançadas e uma view documentada

a) Duas subconsultas escalares:

• Pergunta: Quais são os produtos de preço máximo e quais pedidos estão acima da média de valor de itens?
• SQL:

```
-- Escalar 1: Produto com o maior preço individual
SELECT nome, preco FROM produto WHERE preco = (SELECT MAX(preco) FROM produto);

-- Escalar 2: Pedidos acima da média de valor dos itens
```

SELECT id_pedido, data_pedido FROM pedido WHERE valor_total > (SELECT AVG(quantidade * preco_unitario) FROM item_pedido);

• Expectativa / Resultado Esperado: Escalar 1: Notebook High-End (R$ 8.500,00) | Escalar 2: Pedido #204 (R$ 1.200,00)
• Interpretação: Subconsultas escalares retornam exatamente 1 linha e 1 coluna, agindo como constantes dinâmicas.

b) Duas subconsultas de múltiplas linhas:

• Pergunta: Quais clientes compraram eletrônicos e quais produtos têm preços iguais a qualquer livro cadastrado?
• SQL:

```
-- Múltiplas Linhas 1: Clientes que compraram itens da categoria 'Eletrônicos'
SELECT nome FROM cliente WHERE id_cliente IN (
   SELECT DISTINCT p.id_cliente
   FROM pedido p
   INNER JOIN item_pedido ip ON p.id_pedido = ip.id_pedido
   INNER JOIN produto prod ON ip.id_produto = prod.id_produto
   WHERE prod.categoria = 'Eletrônicos'
);

-- Múltiplas Linhas 2: Produtos com preço correspondente a qualquer preço de 'Livros'
SELECT nome, preco FROM produto WHERE preco IN (
   SELECT preco FROM produto WHERE categoria = 'Livros'
);
```

• Expectativa / Resultado Esperado: Múltiplas 1: Ana Silva, Bruno Lima | Múltiplas 2: Caderno Universitário (R$ 25,00)
• Interpretação: Subconsultas de múltiplas linhas retornam uma coluna com várias linhas e devem ser operadas com IN, ANY ou ALL.

c) Uma solução com EXISTS e outra com NOT EXISTS:

• Pergunta: Quais produtos têm histórico de vendas e quais nunca foram vendidos?
• SQL:

```
-- EXISTS: Produtos que já foram vendidos pelo menos uma vez
SELECT p.id_produto, p.nome
FROM produto p
WHERE EXISTS (
   SELECT 1 FROM item_pedido ip WHERE ip.id_produto = p.id_produto
);

-- NOT EXISTS: Produtos que NUNCA foram vendidos
SELECT p.id_produto, p.nome
FROM produto p
WHERE NOT EXISTS (
   SELECT 1 FROM item_pedido ip WHERE ip.id_produto = p.id_produto
);
```

• Expectativa / Resultado Esperado: EXISTS: Mouse, Teclado, Monitor | NOT EXISTS: Cadeira Ergonomica Premium
• Interpretação: EXISTS/NOT EXISTS testam apenas a existência de correspondência na subconsulta (retornando booleano), interrompendo a busca no primeiro match.

d) Uma subconsulta correlacionada e sua reescrita:

• Pergunta: Qual é o produto mais caro dentro de cada categoria?
• SQL:

```
-- Correlacionada:
SELECT p.nome, p.categoria, p.preco
FROM produto p
WHERE p.preco = (SELECT MAX(p2.preco) FROM produto p2 WHERE p2.categoria = p.categoria);

-- Reescrita eficiente via JOIN com agregação prévia:
SELECT p.nome, p.categoria, p.preco
```

```
FROM produto p
INNER JOIN (
   SELECT categoria, MAX(preco) AS max_preco
   FROM produto
   GROUP BY categoria
) cat_max ON p.categoria = cat_max.categoria AND p.preco = cat_max.max_preco;
```

• Expectativa / Resultado Esperado: Eletrônicos: TV 65' (R$ 4.500,00) | Livros: Enciclopédia (R$ 300,00)
• Interpretação: A reescrita substitui a checagem linha a linha por uma agregação única prévia, otimizando drasticamente o desempenho.

e) Duas CTEs encadeadas para produzir um indicador:

• Pergunta: Qual o total gasto por cada cliente comparado à média geral de vendas de todos os clientes?
• SQL:

```
WITH vendas_cliente AS (
   SELECT p.id_cliente, SUM(ip.quantidade * ip.preco_unitario) AS total_gasto
   FROM pedido p
   INNER JOIN item_pedido ip ON p.id_pedido = ip.id_pedido
   GROUP BY p.id_cliente
),
media_geral AS (
   SELECT AVG(total_gasto) AS media_gasto_clientes
   FROM vendas_cliente
)
SELECT
   vc.id_cliente,
   vc.total_gasto,
   mg.media_gasto_clientes,
   CASE WHEN vc.total_gasto >= mg.media_gasto_clientes THEN 'Acima da Média'
ELSE 'Abaixo da Média' END AS indicador_status
FROM vendas_cliente vc
CROSS JOIN media_geral mg;
```

• Expectativa / Resultado Esperado: Cliente 1: R$ 5.000,00 (Média: R$ 2.100,00) -> Acima da Média | Cliente 2: R$ 800,00 -> Abaixo da Média
• Interpretação: A primeira CTE agrega vendas por cliente; a segunda calcula o indicador médio sobre a primeira, demonstrando encadeamento modular.

f) Uma view que simplifique uma consulta recorrente:

• Pergunta: Como abstrair o cálculo complexo do total e dados de clientes em pedidos recorrentes?
• SQL:

```
CREATE VIEW vw_resumo_pedidos_detalhado AS
SELECT
   p.id_pedido,
   p.data_pedido,
   c.nome AS nome_cliente,
   p.status,
   SUM(ip.quantidade * ip.preco_unitario) AS valor_total_pedido
FROM pedido p
INNER JOIN cliente c ON p.id_cliente = c.id_cliente
INNER JOIN item_pedido ip ON p.id_pedido = ip.id_pedido
GROUP BY p.id_pedido, p.data_pedido, c.nome, p.status;

-- Consulta simples para reutilização:
SELECT * FROM vw_resumo_pedidos_detalhado WHERE status = 'Concluído';
```

• Expectativa / Resultado Esperado: Pedido #101 | 2026-09-10 | Ana Silva | Concluído | R$ 1.550,00
• Interpretação: Encapsula múltiplos JOINs e agregação GROUP BY em um objeto VIEW reutilizável como se fosse uma tabela simples.

g) Uma view que restrinja colunas expostas, com justificativa de privilégios:

• Pergunta: Como disponibilizar dados de clientes para a equipe de suporte sem expor informações sensíveis como CPF, endereço ou dados bancários?
• SQL:

```
CREATE VIEW vw_cliente_contato_publico AS
SELECT
   id_cliente,
   nome,
   email
FROM cliente;

-- Consulta permitida para a equipe de atendimento:
SELECT * FROM vw_cliente_contato_publico;
```

• Expectativa / Resultado Esperado: ID 1 | Ana Silva | ana@email.com
• Interpretação: Justificativa de Privilégios: A View restringe o acesso a colunas confidenciais (LGPD / Segurança). Privilégios de SELECT podem ser concedidos apenas sobre esta VIEW para usuários operacionais, mantendo dados sensíveis bloqueados na tabela base `cliente`.